CIME
Bhubaneswar

उद्यमेन ही सिद्धयन्ति

College of IT & Management Education
Bhubaneswar

ASSIGNMENT

Subject : Programming for problem solving

Topic : Error Handling and Debugging

Name: KULDEEP ROUTRAY

Registration No:

Branch: MCA

Chapter 1: Fundamentals

- 1.1 Introduction to Error Handling
- 1.2 Introduction to Debugging
- 1.3 Key Differences Between Error Handling and Debugging

- **2. Classification of Programming Errors**
- 2.1 Compile-Time Errors (Syntax and Semantic Errors)
- 2.2 Runtime Errors (Exceptions and Environmental Failures)
- 2.3 Resolutions and Defensive Programming Techniques in C
- 2.4 Logical Errors (Flawed Business Logic)

3. Introduction to Debugging

- 3.1 What is Debugging?
- 3.2 The Difference Between Testing, Error Handling, and Debugging

4. Types of Debugging Techniques

- 4.1 Interactive / Breakpoint Debugging
- 4.2 Print-Statement / Logging Debugging
- 4.3 Post-Mortem and Remote Debugging

5. Conclusion

1. Fundamentals

1.1 Introduction to Error Handling

In computer programming, writing executable code is only the first step in software development. Ensuring that a program executes reliably under diverse conditions, copes with unexpected user inputs, and recovers gracefully from execution failures is equally critical. **Error handling** refers to the techniques, strategies, and language mechanisms designed to anticipate, detect, and resolve anomalies or runtime failures.

Without proper error handling, a program encountering an unexpected situation—such as a missing file, invalid input, or memory exhaustion—may crash abruptly, produce corrupted output, or leak system resources.

Core Objectives of Error Handling

1. **Program Stability & Resilience:** Prevent unexpected application crashes and keep the software running in a stable state.
2. **User Experience (UX):** Provide clear, actionable feedback to users instead of cryptic system errors or unhandled call stack dumps.
3. **Resource Safety:** Ensure critical system resources (such as file handles, database connections, and memory blocks) are properly released when failures occur.
4. **Security:** Prevent sensitive system details and stack traces from being exposed to potential attackers.

Key Stages in the Error Life Cycle

- **Detection:** Spotting that an abnormal condition or invalid operation has occurred (e.g., checking if a file pointer returns `NULL` or catching a runtime exception).
- **Reporting/Logging:** Capturing error details, logs, or status codes for developer diagnostics and system audits.
- **Recovery/Mitigation:** Taking corrective action, such as re-prompting the user for valid input, retrying a network operation, falling back to a safe default state, or closing open handles before exiting.

1.2 Introduction to Debugging

While error handling focuses on anticipating and managing potential failures during a program's execution, **debugging** is the methodical, investigative process of identifying, isolating, and rectifying the root cause of those errors or unexpected behaviors within the source code.

In computer programming, a "bug" represents a flaw in logic, syntax, or implementation that causes a program to deviate from its intended specifications. Debugging is arguably one of the most essential skills in software development, often consuming a significant portion of the total engineering lifecycle.

The Debugging Life Cycle

Effective debugging is rarely a matter of random trial and error; it follows a structured scientific method:

1. **Defect Identification:** Recognizing that a bug exists, typically triggered by a compiler error, a runtime crash, or a logical discrepancy between actual output and expected output.
2. **Isolation & Localization:** Narrowing down the scope of the problem from the entire application codebase to a specific module, function, control block, or individual line of code.
3. **Root Cause Analysis:** Examining the state of variables, memory allocations, and execution paths to understand *why* the defect occurred rather than just treating its symptoms.
4. **Correction & Verification:** Modifying the code to fix the core issue, followed by re-testing to ensure the fix resolves the problem without introducing secondary regressions.

1.3 Key Differences Between Error Handling and Debugging

While both **error handling** and **debugging** are essential practices for ensuring software quality and reliability, they serve fundamentally different purposes, occur at different phases of the software lifecycle, and involve different mechanisms.

The table below highlights the key differences between the two concepts:

Feature	Error Handling	Debugging
Primary Purpose	To anticipate, detect, and gracefully manage runtime anomalies and unexpected inputs during program execution.	To identify, isolate, and permanently correct structural, logical, or syntactic defects in the source code.
Lifecycle Phase	**Proactive & Operational:** Built directly into the code to handle events *while the program is running* in production environments.	**Reactive & Developmental:** Performed by developers during the *coding, testing, and maintenance* phases before or after deployment.
Target Audience	**End-Users & Systems:** Provides clear status messages, logging, or fallback states so users aren't met with abrupt crashes.	**Developers:** Provides code-level insights, call stacks, and variable traces to help programmers understand code flaws.
Mechanism	Condition checks (`if-else`), return status codes, try-catch blocks, and exception throw/catch mechanisms.	Code reviews, `printf` tracing, static analyzers, and interactive source-level debuggers (e.g., GDB).
Outcome	Prevents crashes and allows the program to recover or terminate safely without losing critical system integrity.	Modifies and fixes the underlying source code logic so the bug no longer occurs in future executions.

Chapter 2: Types of Errors

2.1 Compile-Time Errors (Syntax & Semantic)

Compile-time errors occur during the translation phase when the compiler converts high-level source code into machine-executable binary. If the source code violates the formal structural or grammatical rules of the programming language, the compiler halts processing and displays diagnostic error messages specifying the line numbers and failure reasons.

Compile-time errors are broadly categorized into **Syntax Errors** and **Semantic Errors**.

A. Syntax Errors

Syntax errors occur when the code violates the strictly enforced grammatical rules of the programming language. The compiler cannot parse or interpret the statements because the structure is invalid.

Key Characteristics

- Caught entirely during compilation (zero runtime risk).
- Execution binary (`.exe` or `.out`) is **not** generated until all syntax errors are resolved.
- Easily localized using standard line-number diagnostics provided by the compiler.

Common Causes

1. **Missing Semicolons:** Omitting `;` at the end of statements in languages like C/C++.
2. **Unmatched Brackets/Quotes:** Opening a brace `{`, parenthesis `(`, or string quote `"` without closing it.
3. **Misspelled Keywords:** Typing `pritnf` or `integar` instead of `printf` or `int`.
4. **Undeclared Variables:** Using a variable name without defining its data type first.

C Code Example (Syntax Error)



B. Semantic Errors

Semantic errors occur when code is grammatically correct according to syntax rules (it compiles without structural syntax flaws), but the statement lacks valid computer-logic meaning or violates operator evaluation standards within the language context.

Unlike syntax errors (which prevent compilation), some semantic issues compile fine but express illegal operations, whereas others (like expression type mismatches or illegal assignments) trigger compile-time semantic warnings/errors.

Operator Precedence (BODMAS / PEMDAS) & Semantic Violations

In arithmetic expressions, programming languages enforce operator precedence (BODMAS: Brackets, Orders, Division/Multiplication, Addition/Subtraction). A common semantic misunderstanding occurs when a programmer writes an expression expecting standard algebraic precedence without properly grouping operators using parentheses.

```c
#include <stdio.h>

int main() {
    int a = 10;
    int b = 20;

    // SEMANTIC / PRECEDENCE VIOLATION:
    // Evaluates as: a + (b / 2) -> 10 + (20 / 2) = 20 (Incorrect Result!)
    int average = a + b / 2;

    // CORRECT SEMANTIC IMPLEMENTATION:
    // Forces addition first: (10 + 20) / 2 = 15
    int correct_average = (a + b) / 2;

    printf("Incorrect Output (BODMAS issue): %d\n", average);
    printf("Correct Output: %d\n", correct_average);

    return 0;
}
```

2.2 Runtime Errors (Exceptions & Environmental Failures)

Runtime errors occur **after successful compilation**, when the program is actively executing. Unlike syntax errors that prevent code from building, runtime errors happen when the program is forced to perform an illegal, undefined, or environmentally constrained operation while processing live data.

When a runtime error occurs, the operating system typically terminates the program abnormally (e.g., *Segmentation Fault*, *Floating Point Exception*), or the application crashes, leading to poor user experience and potential data corruption.

Common Categories of Runtime Errors

1. **Arithmetic Exceptions:** Operations like division by zero or modulo by zero.
2. **Memory Access Faults:** Dereferencing `NULL` pointers or accessing arrays out of bounds.
3. **Environmental Failures:** Missing files, dropped database connections, or unreachable network endpoints.
4. **Invalid User Input:** Supplying non-numeric strings where numbers are expected, or malformed data formats.

Code Examples in C

Below are three standard C examples illustrating runtime errors caused by unexpected user input, missing database connection strings, and missing backend endpoints.

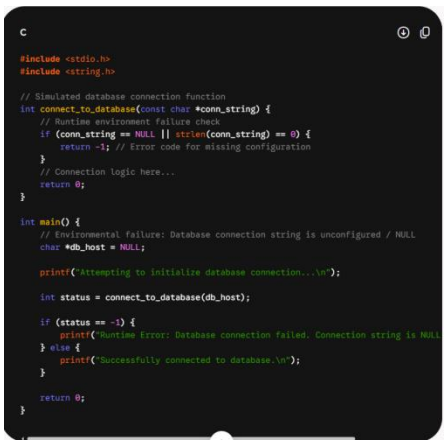
A. Unexpected Input in a Prime Number Check

If a user inputs a non-numeric string, characters, or negative numbers when the program expects a positive integer, the scanning function (`scanf`) fails to parse the data properly, leaving variables uninitialized or causing infinite loops.



B. Missing Database Connection String Example

In systems programming, attempting to connect to a database using an unconfigured, `NULL`, or malformed connection string results in a runtime environment failure.



```c
#include <stdio.h>
#include <string.h>

// Simulated database connection function
int connect_to_database(const char *conn_string) {
    // Runtime environment failure check
    if (conn_string == NULL || strlen(conn_string) == 0) {
        return -1; // Error code for missing configuration
    }
    // Connection logic here...
    return 0;
}

int main() {
    // Environmental failure: Database connection string is unconfigured / NULL
    char *db_host = NULL;

    printf("Attempting to initialize database connection...\n");

    int status = connect_to_database(db_host);

    if (status == -1) {
        printf("Runtime Error: Database connection failed. Connection string is NULL
    } else {
        printf("Successfully connected to database.\n");
    }

    return 0;
}
```

C. Missing Backend Endpoint Example

When a network client attempts to invoke a backend API endpoint that is down, unrouted, or missing (`404 Not Found`), the operation fails at runtime due to environmental unreachability.



```c
#include <stdio.h>
#include <string.h>

// Simulated HTTP request client
int send_http_request(const char *endpoint_url) {
    // Simulating endpoint route registry check
    const char *active_endpoint = "/api/v1/process";

    if (strcmp(endpoint_url, active_endpoint) != 0) {
        return 404; // HTTP 404 Not Found (Missing Endpoint Runtime Failure)
    }
    return 200; // OK
}

int main() {
    // Environmental Failure: Endpoint path has a typo or does not exist on server
    char *target_url = "/api/v1/missing_route";

    printf("Dispatching request to URL: %s\n", target_url);

    int http_status = send_http_request(target_url);

    if (http_status == 404) {
        printf("Runtime Error [HTTP 404]: The requested backend endpoint '%s' was no
    } else {
        printf("Request processed successfully.\n");
    }

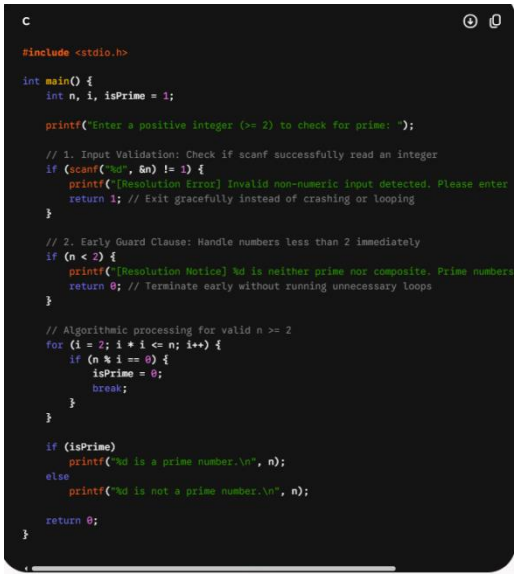
    return 0;
}
```

2.3 Resolutions and Defensive Programming Techniques in C

To mitigate runtime crashes, environmental failures, and unhandled user inputs discussed in previous sections, C programmers implement **defensive programming** and proactive error resolution strategies. Because C lacks native exception handling (`try-catch` blocks), developers use guard clauses, early validation checks, and control-flow patterns to ensure robustness.

A. Resolution for Prime Number Check (Early Guard Clauses & Input Validation)

To prevent runtime crashes from non-numeric strings or invalid numbers, the program must validate input immediately (`scanf` return check) and apply early boundary checks (`n >= 2`) before executing any algorithmic loops.



```c
#include <stdio.h>

int main() {
    int n, i, isPrime = 1;

    printf("Enter a positive integer (>= 2) to check for prime: ");

    // 1. Input Validation: Check if scanf successfully read an integer
    if (scanf("%d", &n) != 1) {
        printf("[Resolution Error] Invalid non-numeric input detected. Please enter
        return 1; // Exit gracefully instead of crashing or looping
    }

    // 2. Early Guard Clause: Handle numbers less than 2 immediately
    if (n < 2) {
        printf("[Resolution Notice] %d is neither prime nor composite. Prime numbers
        return 0; // Terminate early without running unnecessary loops
    }

    // Algorithmic processing for valid n >= 2
    for (i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            isPrime = 0;
            break;
        }
    }

    if (isPrime)
        printf("%d is a prime number.\n", n);
    else
        printf("%d is not a prime number.\n", n);

    return 0;
}
```

B. Resolution for Missing Database & API Strings (Early Configuration Check)

Environmental failures (such as null configuration strings or unreachable backend routes) should be intercepted at the very beginning of application initialization or function execution using **early return guards**.

#include <stdio.h>

#include <string.h>

// Function with early guard check for database connection string

int initialize_database(const char *conn_string) {

```
    // Early validation check
    if (conn_string == NULL || strlen(conn_string) == 0) {
        printf("[Resolution Error] Database configuration string is NULL or empty.\n");
        return -1; // Return failure code immediately
    }

    printf("Connecting to database using string: %s\n", conn_string);
    return 0; // Success
}

// Function with early guard check for API endpoint
int dispatch_api_request(const char *endpoint) {
    const char *valid_endpoint = "/api/v1/process";

    // Early validation check
    if (endpoint == NULL || strcmp(endpoint, valid_endpoint) != 0) {
        printf("[Resolution Error [HTTP 404]]: Target endpoint '%s' is invalid or missing.\n",
               endpoint ? endpoint : "NULL");
        return 404;
    }

    printf("API Request routed successfully to %s\n", endpoint);
    return 200;
```

```
}


int main() {
    // Testing resolutions
    char *db_config = "Host=localhost;Db=app_db;";
    char *api_route = "/api/v1/missing_path";

    // Early validation prevents downstream runtime crashes
    if (initialize_database(db_config) != 0) {
        // Handle database failure gracefully
        return 1;
    }

    dispatch_api_request(api_route);

    return 0;
}
```

C. "Try-Catch" Equivalents in C

Unlike modern object-oriented languages (Java, C++, Python) that use `try-catch-finally` blocks, C handles errors through specific procedural patterns.

1. The `goto cleanup` Pattern (Industry Standard in C)

C developers frequently use a `goto` statement to jump directly to a centralized resource release block when an error occurs, mimicking the cleanup behavior of a `finally` block.

```
#include <stdio.h>
```

```
#include <stdlib.h>

int process_data_file(const char *filename) {
    FILE *file = NULL;
    char *buffer = NULL;
    int error_code = 0;

    // Open file
    file = fopen(filename, "r");
    if (file == NULL) {
        error_code = -1;
        goto cleanup; // Jump straight to resource deallocation
    }

    // Allocate memory buffer
    buffer = (char *)malloc(1024);
    if (buffer == NULL) {
        error_code = -2;
        goto cleanup;
    }

    // Process file logic here...
    printf("File processed successfully.\n");
```

```
cleanup:
    // Equivalent to 'finally' block: Guarantee resource cleanup
    if (file != NULL) {
        fclose(file);
        printf("[Cleanup] File handle closed.\n");
    }
    if (buffer != NULL) {
        free(buffer);
        printf("[Cleanup] Allocated memory freed.\n");
    }

    return error_code;
}

int main() {
    process_data_file("non_existent_file.txt");
    return 0;
}
```

2. Non-Local Jumps (`setjmp` and `longjmp`)

For advanced error handling across nested function calls, C provides `<setjmp.h>`, which acts similarly to throwing and catching exceptions:

- **`setjmp()`**: Saves the current execution context (stack environment) into a buffer.

- **`longjmp()`**: Instantly jumps execution back to the point where `setjmp()` was called, effectively unwinding the stack from deep function calls upon encountering a critical error.

2.4 Logical Errors (Flawed Business Logic)

Logical errors represent the most insidious and difficult-to-detect category of software flaws. A logical error occurs when the source code compiles successfully without any syntax warnings and executes from start to finish without triggering runtime crashes, but **produces incorrect output, flawed calculations, or unexpected business behavior**.

These errors arise entirely from conceptual flaws in the programmer's algorithm, misunderstandings of business requirements, or incorrect implementation of mathematical formulas and control flow structures.

Key Characteristics of Logical Errors

- **Zero Compiler Warnings:** The compiler views the syntax and semantics as perfectly valid, generating a fully executable program (`.exe` or `.out`).
- **Silent Failures:** No exception handlers are triggered, and the system does not crash or throw error codes. The application blindly processes incorrect data and assumes the output is correct.
- **Hardest to Debug:** Because the system provides no diagnostic traces, developers must rely on manual code reviews, test case comparisons, and step-by-step debugging to discover *why* the output deviates from expected results.

Classic C Code Example: Flawed Business Logic

A frequent logical error in business and transactional applications involves accidental use of the assignment operator (=) inside conditional checks instead of the equality operator (==), or incorrect ordering of business rule branches.

Example: Flawed VIP Discount Calculation

Consider an e-commerce checkout module where a customer's VIP status should determine their discount rate.

```c
#include <stdio.h>

int main() {
    int total_amount = 1000;
    int discount = 0;
    int is_vip_customer = 0; // 0 represents False (Regular Customer)

    // LOGICAL ERROR: Using assignment operator '=' instead of equality '=='
    // 'is_vip_customer = 1' assigns 1 to the variable and evaluates to non-zero (Tr
    // bypassing the intended business check entirely!
    if (is_vip_customer = 1) {
        discount = total_amount * 0.20; // 20% VIP Discount
        printf("[Business Logic Executed]: VIP Discount Applied.\n");
    } else {
        discount = total_amount * 0.05; // 5% Regular Discount
        printf("[Business Logic Executed]: Regular Discount Applied.\n");
    }

    printf("Final Payable Amount: %d\n", total_amount - discount);

    return 0;
}
```

Analysis of the Logical Flaw

1. **Intended Business Rule:** If the customer is a VIP (`is_vip_customer == 1`), give a 20% discount; otherwise, give a 5% regular discount.
2. **The Flaw:** Writing `if (is_vip_customer = 1)` mutates the variable state during the condition evaluation. Since the assigned value (`1`) is non-zero, the `if` condition evaluates to `TRUE` regardless of the user's actual status.
3. **The Consequence:** Regular customers incorrectly receive VIP discounts, causing financial discrepancies in production without a single compiler warning.

Resolution and Prevention of Logical Errors

To identify and prevent flawed business logic before software deployment, developers use specialized verification techniques:

1. **Defensive Conditional Coding (Yoda Conditions):** Placing constants on the left-hand side of comparisons (e.g., `if (1 == is_vip_customer)`) causes a compile-time syntax error if a single `=` is accidentally used, preventing the logical bug entirely.
2. **Comprehensive Unit Testing:** Writing automated test suites (using frameworks like Unity or Check in C) with known boundary inputs and expected outputs to verify each modular function independently.
3. **Formal Code Reviews & Peer Walkthroughs:** Having fellow engineers inspect business rules, loop invariants, and boundary conditions to catch logical oversights that the original author missed.

3. Introduction to Debugging

3.1 What is Debugging?

Debugging is the methodical, systematic process of identifying, isolating, and rectifying the root cause of errors, defects, or unexpected behaviors within a program's source code. Unlike error handling—which anticipates and manages runtime anomalies while an application runs—debugging is a developer-driven investigative process. It bridges the gap between observing a system failure (such as an incorrect output or a program crash) and modifying the underlying code so that the defect is permanently resolved.

3.2 The Difference Between Testing, Error Handling, and Debugging

Although often used interchangeably during software development, testing, error handling, and debugging serve three distinct functions across the software lifecycle:

Feature	Software Testing	Error Handling	Debugging
Primary Goal	To discover and verify the presence of defects by running software against expected test inputs and edge cases.	To proactively anticipate, intercept, and gracefully manage runtime failures and invalid inputs during execution.	To locate, analyze, and permanently fix the underlying source code defects causing those failures.
When It Occurs	**Evaluative Phase:** Performed by QA engineers or developers after or during feature implementation.	**Operational Phase:** Built directly into the executing application code in production environments.	**Investigative Phase:** Triggered reactively whenever a test fails, a bug is reported, or a crash occurs.
Mechanism	Test cases, assertions, boundary input suites, and automated test frameworks (e.g., JUnit, Check).	Conditional checks (`if-else`), return status codes, and try-catch mechanisms.	Code walkthroughs, `printf` tracing, static analysis, and interactive debuggers (e.g., GDB).

4. Types of Debugging Techniques

When developers encounter software bugs, they rely on different technical approaches to isolate and solve them depending on the environment and severity of the issue:

4.1 Interactive / Breakpoint Debugging

Interactive debugging utilizes specialized software utilities (such as GDB in C) to pause program execution dynamically at pre-determined lines of code (**breakpoints**).

- **Key Features:** Allows developers to inspect local variable states in real-time, execute code line-by-line (**Step Over** / **Step Into**), and monitor memory allocations or variable mutations (**watchpoints**) without altering the source code.

4.2 Print-Statement / Logging Debugging

Often referred to as trace debugging, this technique involves strategically inserting output commands (such as `printf` in C or `console.log` in higher-level languages) into the source code.

- **Key Features:** Displays intermediate variable values, loop counters, and execution checkpoints directly to the console stream to verify whether execution paths reach expected milestones. While simple, it requires recompiling code for every new trace point.

4.3 Post-Mortem and Remote Debugging

Post-mortem and remote debugging are specialized techniques used when bugs occur in environments where developers lack direct interactive access (such as production servers or embedded devices).

- **Key Features: Post-mortem debugging** analyzes application memory dumps or core dump files *after* a fatal crash has already terminated the program. **Remote debugging** connects a local development debugger over a network socket to a running instance on a remote server or physical device to inspect live execution states.

Conclusion

Mastering **Error Handling and Debugging** is an indispensable milestone for any software developer, forming the absolute foundation of reliable, secure, and production-ready programming. As explored throughout this report within the context of *Programming for Problem Solving (PPS)*:

1. **Proactive Management over Reactive Crashes:** Effective error handling—such as early input validation, boundary guards (e.g., n≥2n \ge 2n≥2), and environmental check routines—prevents brittle applications from failing catastrophically when confronted with unexpected user inputs, missing configuration strings, or unreachable network endpoints.
2. **Understanding Failure Modes:** Recognizing the distinct boundaries between syntax errors, semantic flaws, runtime exceptions, and silent logical errors empowers developers to pinpoint problems before and after compilation.
3. **The Synergy of Error Handling and Debugging:** While robust error handling acts as a dependable operational safety net in deployed environments, systematic debugging techniques—ranging from trace logging and interactive GDB breakpoints to post-mortem analysis—provide the developer tools necessary to isolate root causes and permanently refactor flawed source code.

Ultimately, transitioning from naive coding to defensive, disciplined engineering ensures that software systems maintain high availability, data integrity, and resilience against the unpredictable conditions of real-world execution.